الطبقات والجدران في معمار الخلّان

Lecture 4 — Web Application Architecture

> **Three-tier (N-tier) architecture** : the application split into a **presentation** tier (what the user sees), an **application** tier (business logic) and a **data** tier (persistent storage).

The split is a **security boundary**: the presentation tier never talks to the database directly; every request passes through the application tier's logic first.

Component	Role	Key security consideration
Reverse proxy / Load balancer	Distributes traffic, often terminates TLS	Forward the real client IP; handle the **Host** header safely
API gateway	Single entry point for backend APIs	Good place for auth and rate limiting — but one misconfiguration exposes everything behind it
Application server	Executes business logic	Where most **injection** vulnerabilities live
Cache (e.g. Redis)	Speeds up repeated reads	Can leak data across users if not access-controlled per session
CDN	Serves static assets from edge locations	Cache poisoning / deception risk
Database	Persistent storage	Never directly reachable from the internet; least-privilege accounts
Third-party APIs	Payments, maps, analytics	Expands attack surface beyond your code — **supply-chain** risk

> **MVC (Model-View-Controller)** : **Model** holds data and business rules; **View** renders what the user sees; **Controller** receives requests, decides what happens, coordinates the other two.

Q/ Where do input validation and authorization checks belong in MVC ?

- In the **Controller and Model**, before any business logic runs — **never** left to the View, never assumed done upstream.

> **Trust boundary** : any point where data crosses from a zone you don't control into one you do, or from lower to higher privilege.

The browser and the network are **always untrusted**. Inside the app tier, data still isn't safe for the database — cross that boundary with **parameterized queries**.

STRIDE threat modeling

Q/ What are the six STRIDE categories ?

1. **S**poofing — pretending to be someone/something else.
2. **T**ampering — altering data in transit or at rest.
3. **R**epudiation — denying an action with no proof otherwise.
4. **I**nformation disclosure — exposing data to the wrong people.
5. **D**enial of service — making the system unavailable.
6. **E**levation of privilege — gaining more access than granted.

> **Threat modeling** : walking the architecture **boundary by boundary**, asking which STRIDE category applies and what the defense is — **before** the system is built.

Boundary	Likely STRIDE threat	Impact & primary defense
Client → CDN / Proxy	Tampering, DoS	Traffic altered or flooded → TLS everywhere, rate limiting, WAF
Proxy → App server	Spoofing, Info disclosure	Forged client IP or Host header → validate X-Forwarded-For, allow-list domains
App server → Database	Tampering, Elevation	SQLi or over-privileged DB account → parameterized queries + least privilege
App → Other microservices	Spoofing, Elevation	"Internal = safe" assumption → **mTLS + Zero Trust**

Attacks on intermediaries

> **Web Cache Poisoning** : abuses **unkeyed inputs** (headers/parameters that change the response but aren't part of the cache key, e.g. **X-Forwarded-Host**). One malicious request poisons the cached copy served to **everyone** afterwards.

> **Web Cache Deception** : tricks the cache into storing a **private** page. A request like `/account/profile.css` hits a dynamic, authenticated endpoint, but the cache — fooled by the `.css` suffix — stores the personal page as a static asset, retrievable by anyone requesting the same URL.

Cache poisoning	Cache deception
Poisons shared content for everyone	Exposes **one victim's private** content
Abuses an unkeyed input	Abuses a static-looking extension / path mapping
Attacker's response reaches victims	Victim's response reaches the attacker

> **HTTP Request Smuggling** : a request carries both **Content-Length** and **Transfer-Encoding: chunked**, and the proxy trusts one while the backend trusts the other (classic **CL.TE**: proxy uses CL, backend uses TE). A second, hidden request reaches the backend **without passing the proxy's filters**.

Fix: keep proxy and backend parsing **strictly consistent**; prefer **HTTP/2 end-to-end**.

Monolith, microservices and Zero Trust

Monolith	Microservices
One deployable unit	Many small, independently deployed services
Simpler to build	Limits how far one compromised service reaches
One vulnerability can expose everything (one blast radius)	Many more network boundaries, each needs auth and validation

> **Zero Trust (ZTNA)** : no request is trusted because of where it came from — every service-to-service call is **authenticated and authorized every time**.

> **mTLS (mutual TLS)** : both sides present certificates, so Service A proves who it is to Service B **and vice versa** (normal HTTPS: only the server proves itself).

> **Service mesh** : an infrastructure layer that automatically wraps every internal call in mTLS.

> **SSRF (Server-Side Request Forgery, A10:2021)** : tricking the app server into requesting an **internal-only** address; internal services that "trust the inside" respond — including cloud metadata endpoints like **169.254.169.254** that can hand over credentials.

Control: forbid outbound requests to internal ranges and metadata addresses unless allow-listed.

Database tier

Account	Privileges	Used by
App read account	SELECT only	Reports, search, read-heavy endpoints
App write account	SELECT, INSERT, UPDATE (no DROP/ALTER)	Core transactional logic (checkout, profile)
Migration / admin	Full schema privileges	CI/CD migration jobs only — **never** the running app

Never connect as a superuser; configure **connection pools per account type** so a compromised read path can't borrow write credentials.

> **Insecure Design (A04:2021)** : flaws baked into the architecture before any code is written. A well-coded implementation of a badly designed architecture is still insecure.

Q/ Booking/payment app — apply the lecture's rules.

1. Redis (sessions) on a **private segment**, reachable only from the app tier.
2. Payment API keys and DB credentials in a **secrets manager**, injected at startup, never hard-coded or returned.
3. App server **forbidden** from requesting internal IPs / metadata (anti-SSRF).
4. Internal call to the payment microservice over **mTLS** (Zero Trust).

Summary of Web Applications Security L4: Web Application Architecture. Built only from the lecture file; check the original slides for anything you're unsure about.